

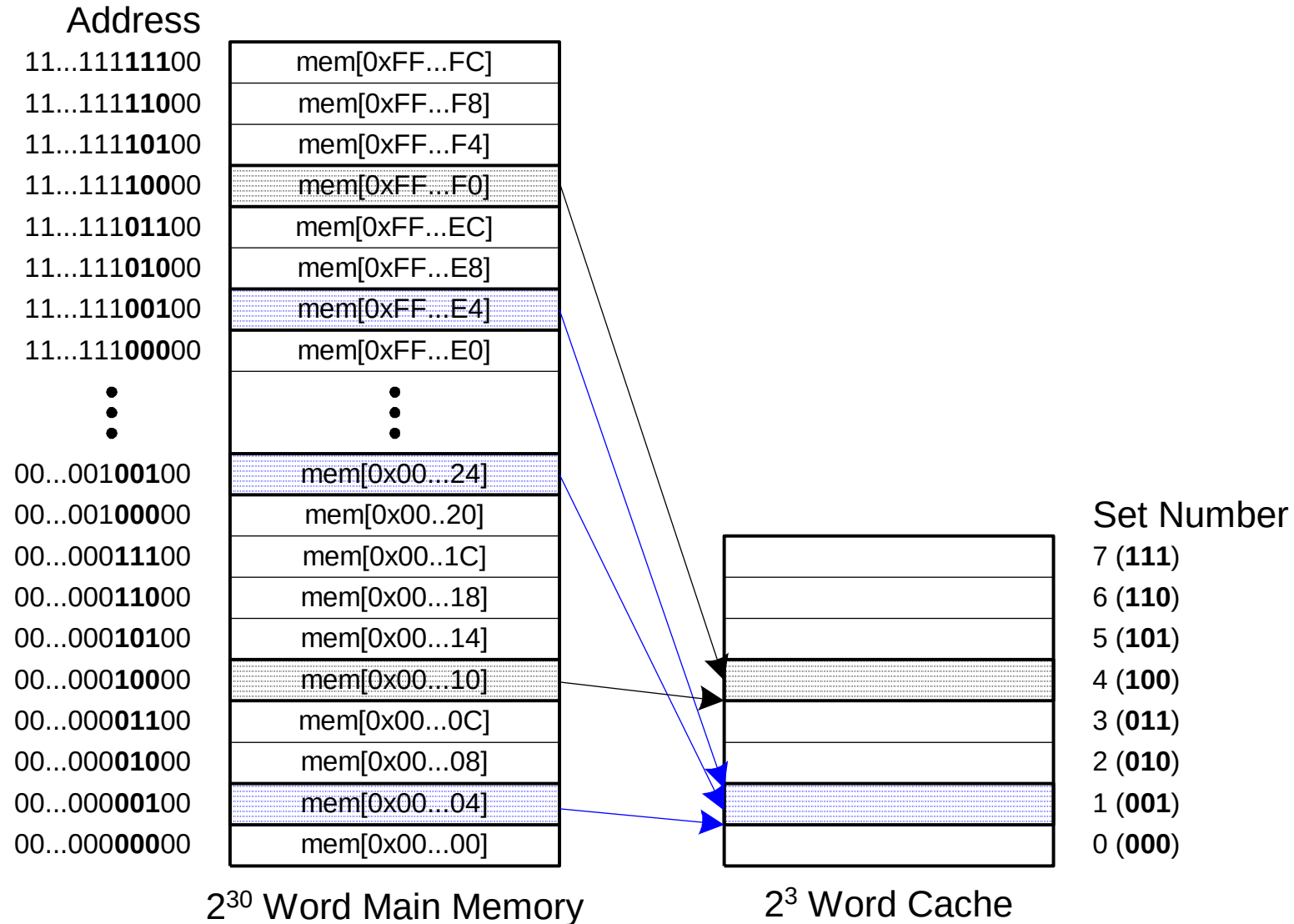
Cache Basics and Operation

Readings

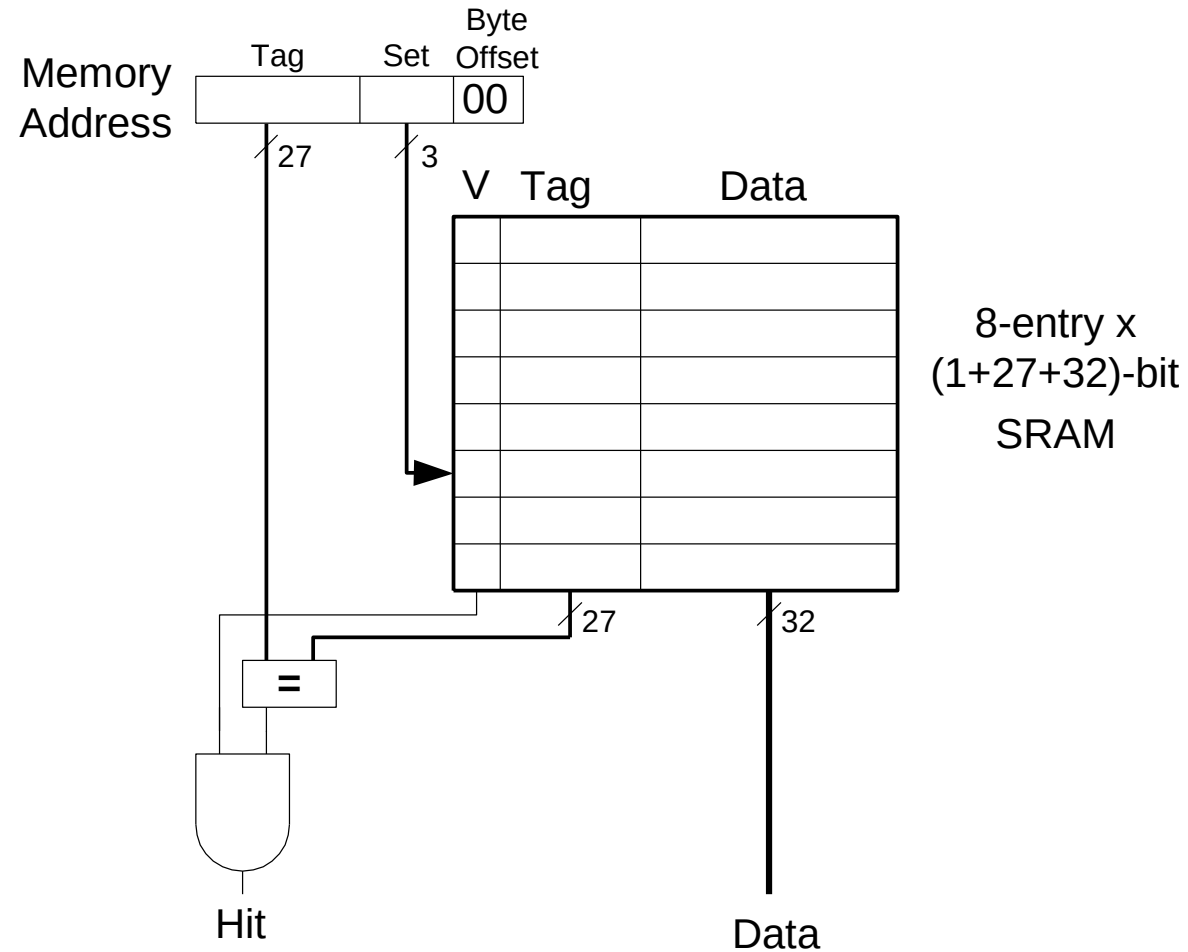
- *Digital Design and Computer Architecture – David Harris & Sarah Harris*
 - Chapter 8

Cache Examples

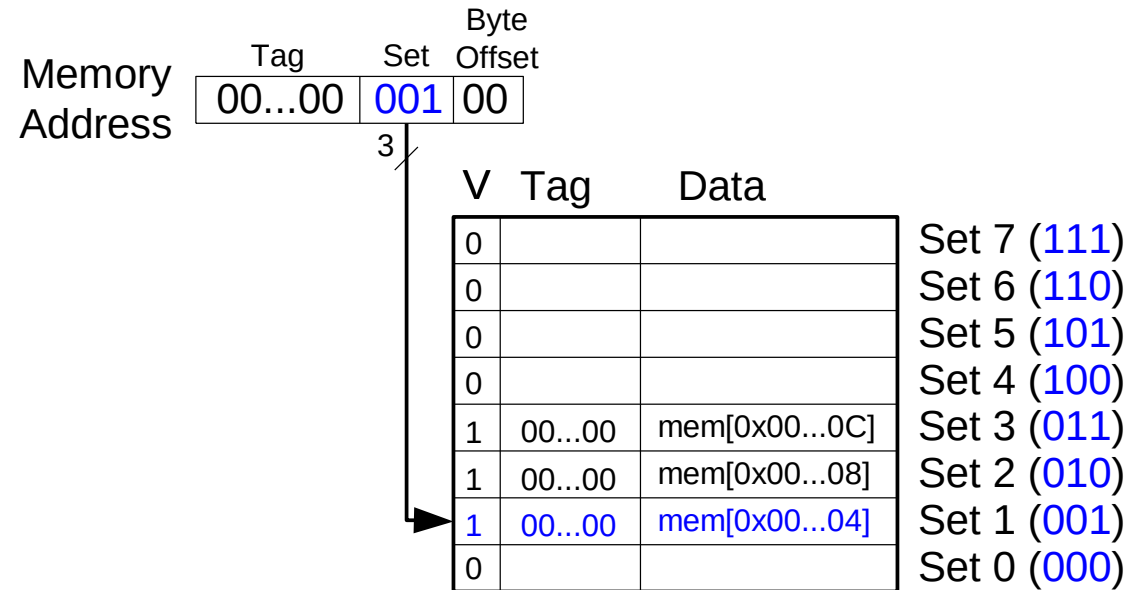
Direct Mapped Cache



Direct Mapped Cache Hardware



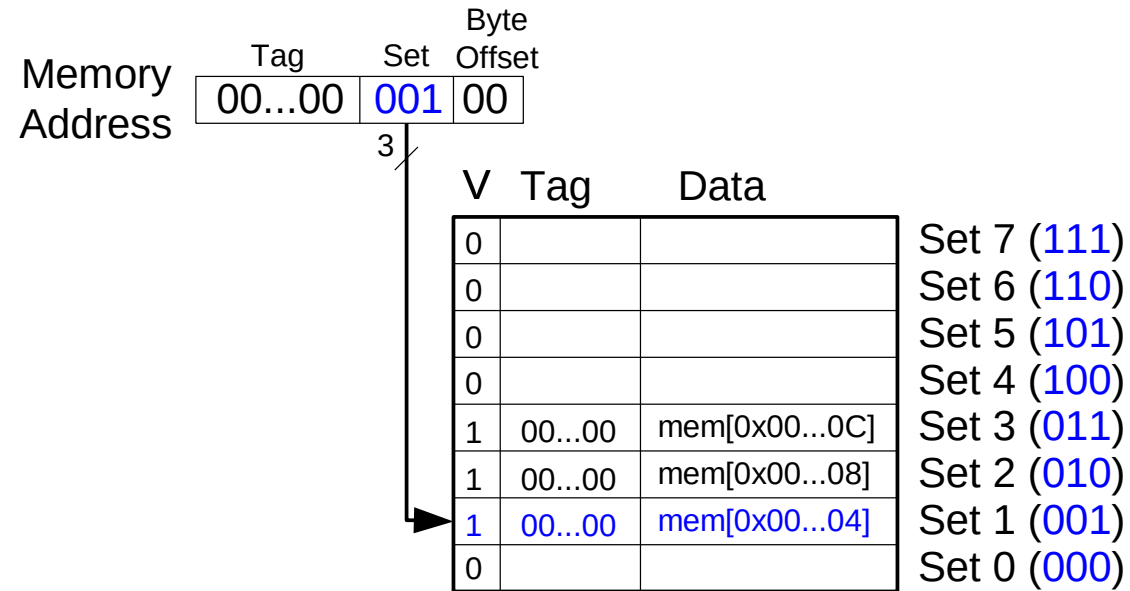
Direct Mapped Cache Performance



```
# MIPS assembly code
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0xC($0)
      lw   $t3, 0x8($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate =

Direct Mapped Cache Performance



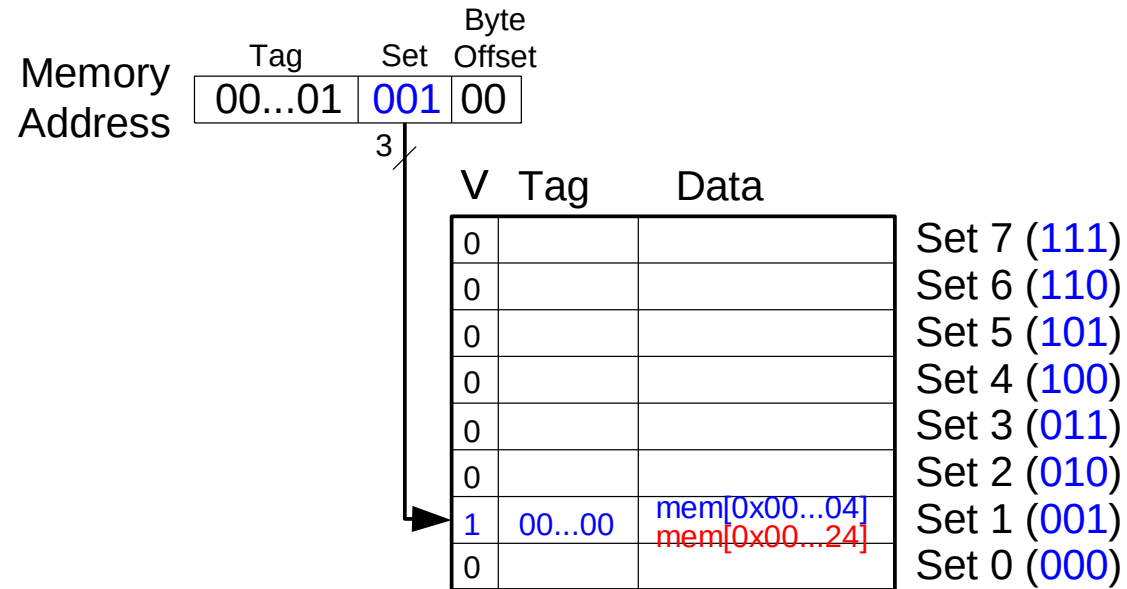
```
# MIPS assembly code
loop:   addi $t0, $0, 5
        beq  $t0, $0, done
        lw   $t1, 0x4($0)
        lw   $t2, 0xC($0)
        lw   $t3, 0x8($0)
        addi $t0, $t0, -1
        j    loop
done:
```

$$\text{Miss Rate} = 3/15 =$$

20%

Temporal Locality
Compulsory Misses

Direct Mapped Cache: Conflict



MIPS assembly code

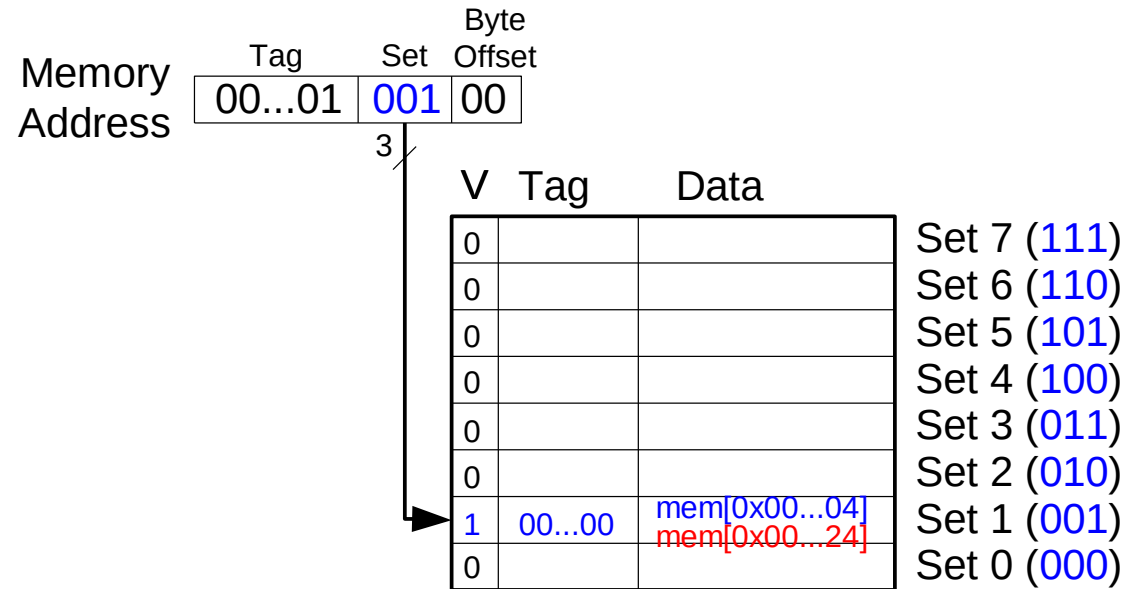
```

    addi $t0, $0, 5
loop: beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:

```

Miss Rate =

Direct Mapped Cache: Conflict

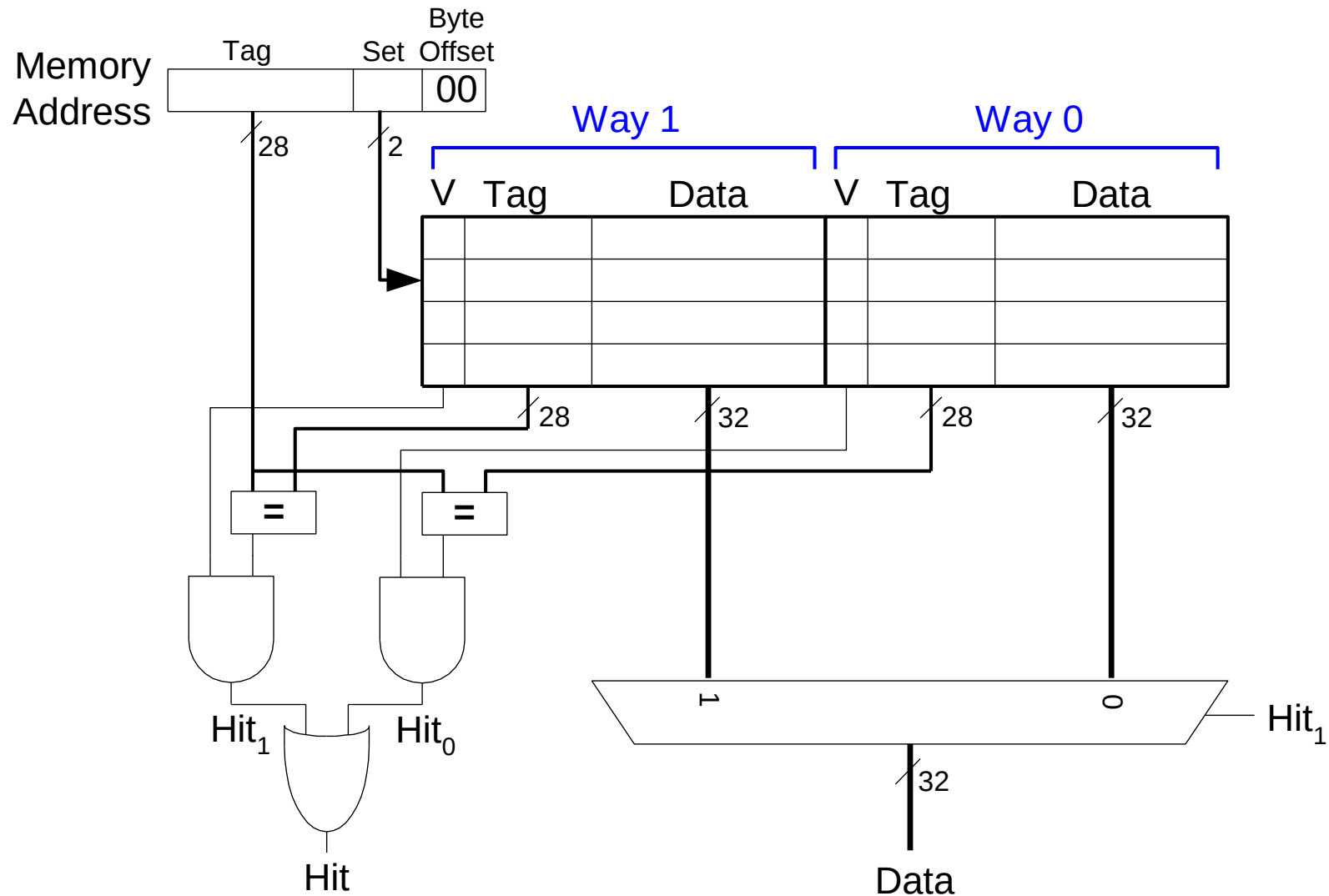


```
# MIPS assembly code
      addi $t0, $0, 5
loop: beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

*Miss Rate = 10/10
= 100%*

Conflict Misses

N-Way Set Associative Cache



N-way Set Associative Performance

MIPS assembly code

```
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate =

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...10	mem[0x00...24]	1	00...00	mem[0x00...04]	Set 1
0			0			Set 0

N-way Set Associative Performance

MIPS assembly code

```
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate = 2/10
= 20%

Associativity reduces
conflict misses

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...10	mem[0x00...24]	1	00...00	mem[0x00...04]	Set 1
0			0			Set 0

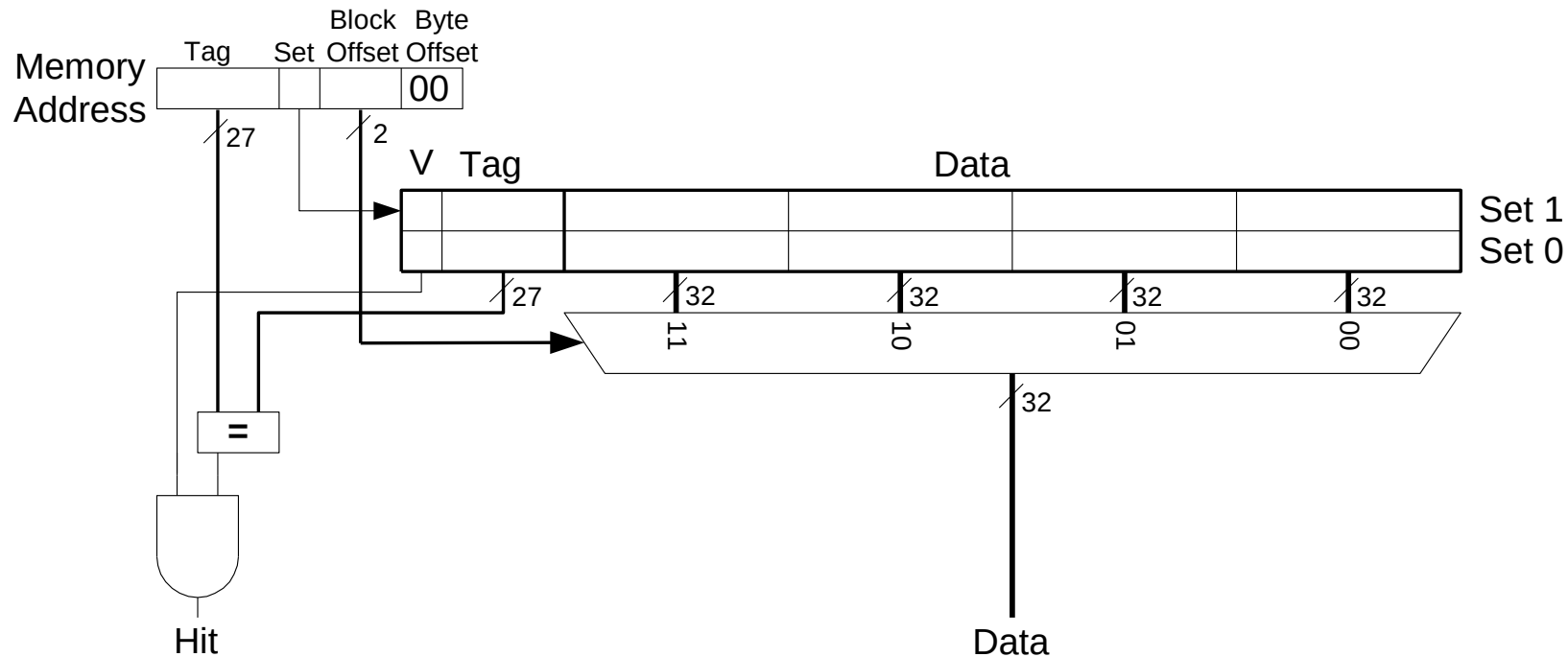
Fully Associative Cache

- No conflict misses
- Expensive to build

[illegible]

Spatial Locality?

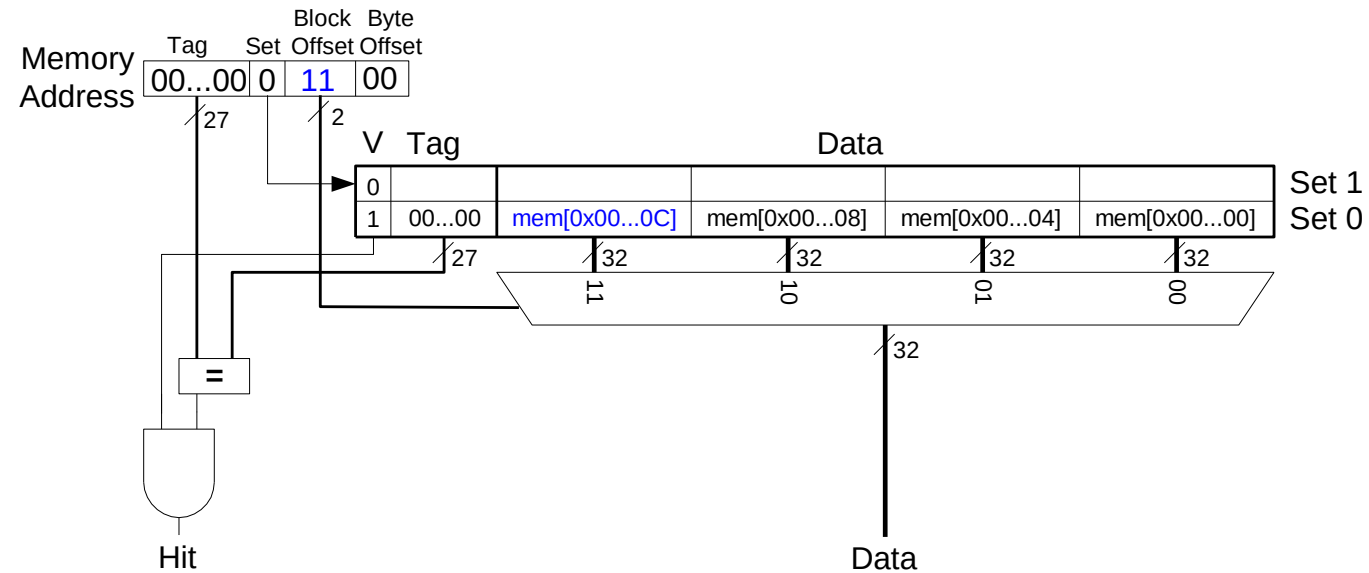
- Increase block size:
 - Block size, $b = 4$ words
 - $C = 8$ words
 - Direct mapped (1 block per set)
 - Number of blocks, $B = C/b = 8/4 = 2$



Direct Mapped Cache Performance

```
      addi $t0, $0, 5
loop:  beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0xC($0)
      lw   $t3, 0x8($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate =

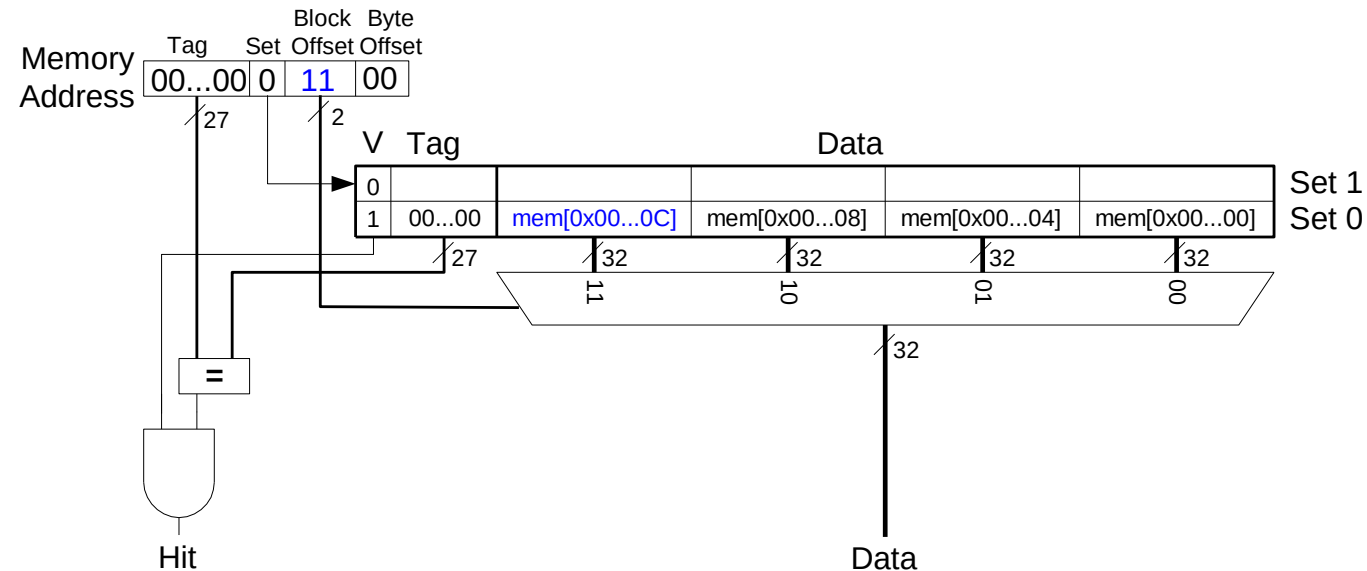


Direct Mapped Cache Performance

```
loop:  addi $t0, $0, 5
      beq $t0, $0, done
      lw  $t1, 0x4($0)
      lw  $t2, 0xC($0)
      lw  $t3, 0x8($0)
      addi $t0, $t0, -1
      j   loop
done:
```

Miss Rate = 1/15
= 6.67%

Larger blocks reduce compulsory misses through spatial locality



Types of Misses

- **Compulsory**: first time data is accessed
- **Capacity**: cache too small to hold all data of interest
- **Conflict**: data of interest maps to same location in cache
- **Miss penalty**: time it takes to retrieve a block from lower level of hierarchy

Capacity Misses

- Cache is too small to hold all data of interest at one time
 - If the cache is full and program tries to access data X that is not in cache, cache must evict data Y to make room for X
 - **Capacity miss** occurs if program then tries to access Y again
 - X will be placed in a particular set based on its address
- In a **direct mapped** cache, there is only one place to put X
- In an **associative cache**, there are multiple ways where X could go in the set.
- How to choose Y to minimize chance of needing it again?
 - Least recently used (LRU) replacement: the least recently used block in a set is evicted when the cache is full.

Cach block size and cach associativity

Cache block size and cache associativity

- Cache block size (8, 16, 32, 64, or 128 Bytes)↵
- Cache associativity (1, 2, or 4-way)↵

The only statistic that you can collect on this system is cache hit rate after performing a sequence of memory accesses. Here is what you observe: ↵

Addresses Accessed (Oldest -> Youngest): 8 times

Hit Rate↵

31	8192	63	25	16384	4096	8192	64	16384	5/9↵
----	------	----	----	-------	------	------	----	-------	------

Cache block size and cache associativity

- Cache block size (8, 16, 32, 64, or 128 Bytes)↵
- Cache associativity (1, 2, or 4-way)↵

The only statistic that you can collect on this system is cache hit rate after performing a sequence of memory accesses. Here is what you observe: ↵

Addresses Accessed (Oldest -> Youngest): 8 times

Hit Rate↵

31	8192	63	25	16384	4096	8192	64	16384	5/9↵
----	------	----	----	-------	------	------	----	-------	------

What is the size of cache block? Circle one: 8B / 16B / 32B / 64B / 128B↵

What is the associativity of the cache? Circle one: 1-way / 2-way / 4-way↵

Cache Replacement

Issues in Set-Associative Caches

- Think of each block in a set having a “priority”
 - Indicating how important it is to keep the block in the cache
- Key issue: How do you determine/adjust block priorities?
- There are three key decisions in a set:
 - Insertion, promotion, eviction (replacement)
- Insertion: What happens to priorities on a cache fill?
 - Where to insert the incoming block, whether or not to insert the block
- Promotion: What happens to priorities on a cache hit?
 - Whether and how to change block priority
- Eviction/replacement: What happens to priorities on a cache miss?
 - Which block to evict and how to adjust priorities

Eviction/Replacement Policy

- Which block in the set to replace on a cache miss?
 - Any invalid block first
 - If all are valid, consult the replacement policy
 - Random
 - FIFO
 - Least recently used (how to implement?)
 - Not most recently used
 - Least frequently used?
 - Least costly to re-fetch?
 - Why would memory accesses have different cost?
 - Hybrid replacement policies
 - Optimal replacement policy?

Implementing LRU

- Idea: Evict the least recently accessed block
- Problem: Need to keep track of access ordering of blocks
- Question: 2-way set associative cache:
 - What do you need to implement LRU perfectly?
- Question: 4-way set associative cache:
 - What do you need to implement LRU perfectly?
 - How many different orderings possible for the 4 blocks in the set?
 - How many bits needed to encode the LRU order of a block?
 - What is the logic needed to determine the LRU victim?

Approximations of LRU

- Most modern processors do not implement “true LRU” (also called “perfect LRU”) in highly-associative caches
- Why?
 - True LRU is complex
 - LRU is an approximation to predict locality anyway (i.e., not the best possible cache management policy)
- Examples:
 - Not MRU (not most recently used)
 - Hierarchical LRU: divide the N-way set into M “groups”, track the MRU group and the MRU way in each group
 - Victim-NextVictim Replacement: Only keep track of the victim and the next victim

Cache Replacement Policy: LRU or Random

- LRU vs. Random: Which one is better?
 - Example: 4-way cache, cyclic references to A, B, C, D, E
 - 0% hit rate with LRU policy
- **Set thrashing**: When the “program working set” in a set is larger than set associativity
 - Random replacement policy is better when thrashing occurs
- In practice:
 - Depends on workload
 - Average hit rate of LRU and Random are similar

Handling Writes (I)

- When do we write the modified data in a cache to the next level?
 - Write through: At the time the write happens
 - Write back: When the block is evicted
- Write-back
 - + Can combine multiple writes to the same block before eviction
 - Potentially saves bandwidth between cache levels + saves energy
 - Need a bit in the tag store indicating the block is “dirty/modified”
- Write-through
 - + Simpler
 - + All levels are up to date. Consistency: Simpler cache coherence because no need to check close-to-processor cache's tag stores for presence
 - More bandwidth intensive; no combining of writes

What's In A Tag Store Entry?

- Valid bit
- Tag
- Replacement policy bits
- Dirty bit?
 - Write back vs. write through caches

Instruction vs. Data Caches

- Separate or Unified?
- Pros and Cons of Unified:
 - + Dynamic sharing of cache space: no overprovisioning that might happen with static partitioning (i.e., separate I and D caches)
 - Instructions and data can thrash each other (i.e., no guaranteed space for either)
 - I and D are accessed in different places in the pipeline. Where do we place the unified cache for fast access?
- First level caches are almost always split
 - Mainly for the last reason above
- Higher level caches are almost always unified

Multi-level Caching in a Pipelined Design

- First-level caches (instruction and data)
 - Decisions very much affected by cycle time
 - Small, lower associativity; latency is critical
 - Tag store and data store accessed in parallel
- Second-level caches
 - Decisions need to balance hit rate and access latency
 - Usually large and highly associative; latency not as important
 - Tag store and data store accessed serially

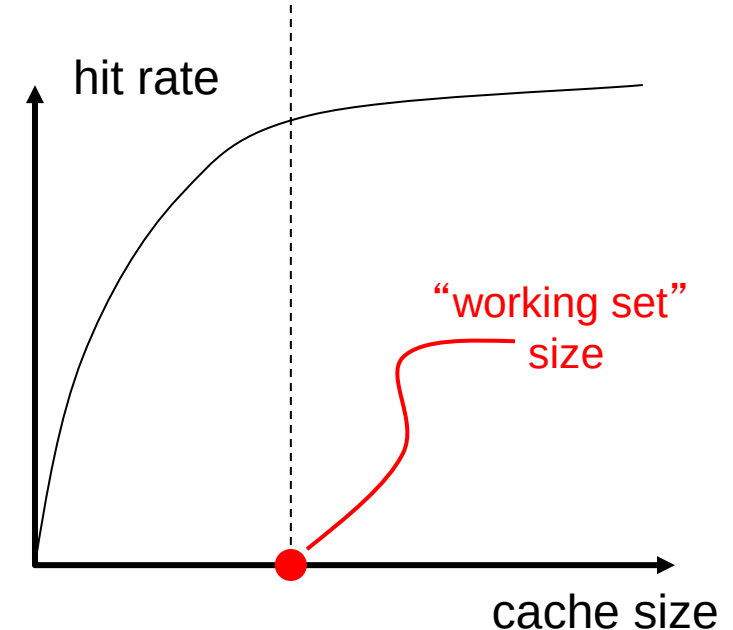
Cache Performance

Cache Parameters vs. Miss/Hit Rate

- Cache size
- Block size
- Associativity
- Replacement policy
- Insertion/Placement policy

Cache Size

- Cache size: total data (not including tag) capacity
 - bigger can exploit temporal locality better
 - not ALWAYS better
- **Too large** a cache adversely affects hit and miss latency
 - smaller is faster => bigger is slower
 - access time may degrade critical path
- **Too small** a cache
 - doesn't exploit temporal locality well
 - useful data replaced often
- **Working set**: the whole set of data the executing application references
 - Within a time interval



Block Size

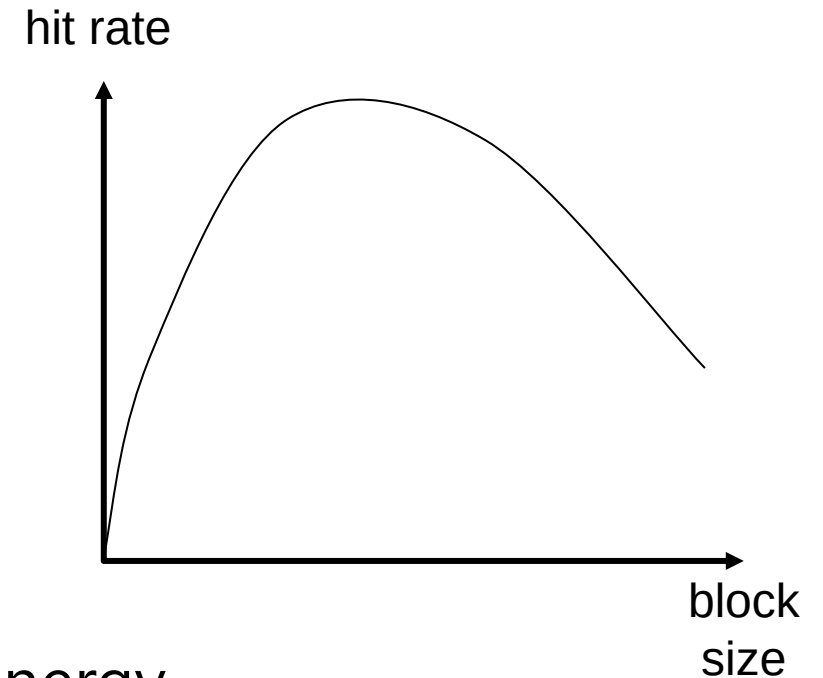
- Block size is the data that is associated with an address tag
 - not necessarily the unit of transfer between hierarchies
 - Sub-blocking: A block divided into multiple pieces (each w/ V/D bits)

- **Too small** blocks

- don't exploit spatial locality well
 - have larger tag overhead

- **Too large** blocks

- too few total # of blocks $\rightarrow$ less temporal locality exploitation
 - waste of cache space and bandwidth/energy if spatial locality is not high



Associativity

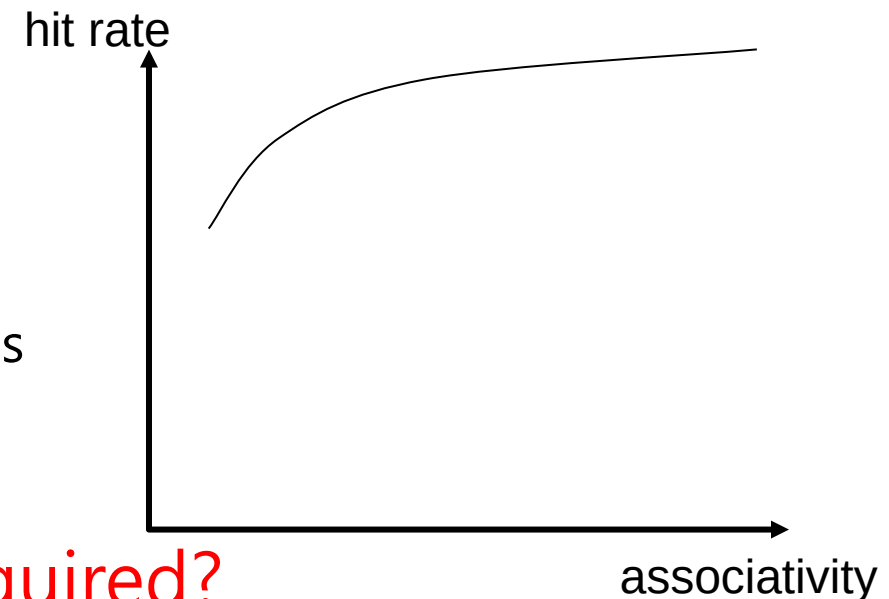
- How many blocks can be present in the same index (i.e., set)?

- Larger associativity

- lower miss rate (reduced conflicts)
- higher hit latency and area cost (plus diminishing returns)

- Smaller associativity

- lower cost
- lower hit latency
 - Especially important for L1 caches



- Is power of 2 associativity required?

Classification of Cache Misses

- Compulsory miss

- first reference to an address (block) always results in a miss
- subsequent references should hit unless the cache block is displaced for the reasons below

- Capacity miss

- cache is too small to hold everything needed
- defined as the misses that would occur even in a fully-associative cache (with optimal replacement) of the same capacity

- Conflict miss

- defined as any miss that is neither a compulsory nor a capacity miss

How to Reduce Each Miss Type

- **Compulsory**
 - Caching cannot help
 - Prefetching can: Anticipate which blocks will be needed soon
- **Conflict**
 - More associativity
 - Other ways to get more associativity without making the cache associative
 - Victim cache
 - Better, randomized indexing
- **Capacity**
 - Utilize cache space better: keep blocks that will be referenced
 - Software management: divide working set and computation such that each “computation phase” fits in cache

Improving Basic Cache Performance

■ Reducing miss rate

- ❑ More associativity
- ❑ Alternatives/enhancements to associativity
 - Victim caches, hashing, pseudo-associativity, skewed associativity
- ❑ Better replacement/insertion policies
- ❑ Software approaches

■ Reducing miss latency/cost

- ❑ Multi-level caches
- ❑ Critical word first
- ❑ Subblocking/sectoring
- ❑ Better replacement/insertion policies
- ❑ Non-blocking caches (multiple cache misses in parallel)
- ❑ Multiple accesses per cycle
- ❑ Software approaches

Software Approaches

Software Approaches for Higher Hit Rate

- Restructuring data access patterns
- Restructuring data layout

- Loop interchange
- Data structure separation/merging
- Blocking
- ...

Restructuring Data Access Patterns (I)

- Idea: Restructure data layout or data access patterns
- Example: If column-major
 - $x[i+1,j]$ follows $x[i,j]$ in memory
 - $x[i,j+1]$ is far away from $x[i,j]$

Poor code

```
for i = 1, rows
  for j = 1, columns
    sum = sum + x[i,j]
```

Better code

```
for j = 1, columns
  for i = 1, rows
    sum = sum + x[i,j]
```

- This is called **loop interchange**
- Other optimizations can also increase hit rate
 - Loop fusion, array merging, ...

Restructuring Data Access Patterns (II)

- **Blocking**
 - Divide loops operating on arrays into computation chunks so that each chunk can hold its data in the cache
 - Avoids cache conflicts between different chunks of computation
 - Essentially: Divide the working set so that each piece fits in the cache
- Also called Tiling

Restructuring Data Layout (I)

```
struct Node {  
    struct Node* next;  
    int key;  
    char [256] name;  
    char [256] school;  
}  
  
while (node) {  
    if (node→key == input-key) {  
        // access other fields of node  
    }  
    node = node→next;  
}
```

- Pointer based traversal (e.g., of a linked list)
- Assume a huge linked list (1B nodes) and unique keys
- Why does the code on the left have poor cache hit rate?
 - “Other fields” occupy most of the cache line even though rarely accessed!

Restructuring Data Layout (II)

```
struct Node {  
    struct Node* next;  
    int key;  
    struct Node-data* node-data;  
}
```

```
struct Node-data {  
    char [256] name;  
    char [256] school;  
}
```

```
while (node) {  
    if (node→key == input-key) {  
        // access node→node-data  
    }  
    node = node→next;  
}
```

- Idea: separate frequently-used fields of a data structure and pack them into a separate data structure
- Who should do this?
 - Programmer
 - Compiler
 - Profiling vs. dynamic
 - Hardware?
 - Who can determine what is frequently used?